

Combinatorial Decision Making and Optimization

Exercise-Type Reference Sheet

*A map of the exercise types that can appear in the written exam, organized by area,
each with a short worked example and a strategy for approaching it under time pressure.*

Contents

1	Constraint Programming (CP)	2
1.1	Modelling	2
1.2	Propagation	6
1.3	Search	8
2	SAT	11
2.1	Encoding pseudo-Boolean / cardinality constraints	11
2.2	Direct SAT Encoding of a CSP	13
2.3	Solving: Resolution and DPLL	14
3	SMT	17
3.1	Theories and CDCL(T)	17
3.2	Theory of Bit-Vectors	21
3.3	Theory of Arrays	21
3.4	Lazy SMT: the abstraction-refinement loop	23
3.5	Clause Learning in SAT/SMT (CDCL / CDCL(T))	24
4	LP / ILP / MIP	29
4.1	Modelling	29
4.2	Simplex, Gomory Cuts, and Branch-and-Bound (numeric)	31
4.3	Duality and Sensitivity	33

1 Constraint Programming (CP)

1.1 Modelling

How to approach it:

- Always structure the answer in three labelled parts: **Variables and Domains**, **Constraints**, **Objective** (if any). Comment each constraint in one line.
- First find the “obvious” set of decision variables (what am I choosing?), then check whether a compact domain exists.
- If comparing two models, always argue in terms of: number of constraints, arity of constraints, and whether a global constraint applies.
- Never forget *implicit* real-world constraints not stated explicitly in the text (e.g., “a resource cannot be used twice at once”).

Type 1 — Basic modelling from scratch

Assignment

A tournament has 6 teams. Each team plays every other team exactly once, scheduled over 5 match days. Each day can host at most 3 matches (3 fields available). Model the problem: define variables, domains, and constraints.

Source: practice exercise constructed for this guide (not from official course material), following the modelling style of *Modelling.pdf*

Solution

Step 1 — decide what a variable represents. There are $\binom{6}{2} = 15$ distinct matches (one per pair of teams). Each match needs a day, so create one variable per match: G_{ij} for $1 \leq i < j \leq 6$, meaning “team i plays team j on day G_{ij} ”. Domain: $D(G_{ij}) = \{1, \dots, 5\}$ (5 match days).

Step 2 — the explicit constraint (field capacity). At most 3 matches per day, so count how many variables equal each day value:

$$\forall g \in \{1, \dots, 5\} : |\{(i, j) : G_{ij} = g\}| \leq 3$$

Step 3 — the implicit constraint. The text never says a team cannot play twice on the same day, but this is physically necessary. For a fixed team k , its 5 matches are the variables G_{ij} where $i = k$ or $j = k$; these must all fall on different days:

$$\forall k = 1, \dots, 6 : \text{alldifferent}(\{G_{ij} : i = k \text{ or } j = k\})$$

Full model: the variables/domains of Step 1, plus the two constraint families of Steps 2–3.

Type 2 — Comparing alternative models

Assignment

Consider the Golomb Ruler problem: place m marks on a ruler such that the distance between every pair of marks is different, and the ruler length is minimum. A naive model uses variables $X_1, \dots, X_m$ (mark positions) with constraint: for all $i_1 < j_1, i_2 < j_2, (i_1, j_1) \neq (i_2, j_2)$: $|X_{i_1} - X_{j_1}| \neq |X_{i_2} - X_{j_2}|$. **(i)** Explain why this model is problematic. **(ii)** Propose an improved model using auxiliary variables. **(iii)** Propose a further improvement using symmetry breaking.

Source: *Modelling.pdf* (Golomb Ruler example)

Solution

(i) Why it's problematic.

- Count the distances first: with m marks there are $\binom{m}{2} = O(m^2)$ pairs, hence $O(m^2)$ distances $|X_i - X_j|$.
- The naive constraint doesn't just list these distances — it compares every *pair* of them (to check they're different), so the number of constraints is $O(m^2)$ distances $\times O(m^2)$ distances $= O(m^4)$.
- Each single constraint mentions up to 4 different X variables ($X_{i_1}, X_{j_1}, X_{i_2}, X_{j_2}$) — a *quaternary* constraint. Quaternary constraints are both numerous here and weak to propagate (a solver struggles to prune domains from them).

(ii) Improved model. The core problem is that “distance” is never given its own variable — it's recomputed inside every constraint. Fix: introduce one auxiliary variable per distance,

$$D_{ij} = |X_i - X_j| \quad \text{for every } i < j$$

Now “all distances differ” is a single global constraint over these auxiliary variables:

$$\text{alldifferent}([D_{12}, D_{13}, \dots, D_{(m-1)m}])$$

This drops the constraint count from $O(m^4)$ to $O(m^2)$ (the $D_{ij} = |X_i - X_j|$ definitions, each *ternary*) plus a single **alldifferent**, which has dedicated matching-based propagation instead of blind pairwise checks.

(iii) Symmetry breaking. Any solution can be mirrored (positions reversed) or shifted, giving equivalent rulers. Fix the order of the marks:

$$X_1 < X_2 < \dots < X_m$$

Two side benefits, not just symmetry removal: (a) **alldifferent**(X) becomes redundant, since strictly increasing values are automatically all different; (b) the absolute value is no longer needed, since we now know $X_i < X_j$ for $i < j$: $D_{ij} = X_j - X_i$.

Type 3 — Global constraints

Assignment

5 employees must each cover exactly one of 5 distinct shifts in a day (no shift uncovered, no employee double-booked). Express this first using `alldifferent`, then rewrite it in “elementary” form (binary $\neq$ constraints).

Source: practice exercise constructed for this guide (not from official course material), illustrating the *alldifferent* decomposition principle from *PropagationGlobalConstraints.pdf*

Solution

Step 1 — variables. $E_i \in \{1, \dots, 5\}$ for $i = 1, \dots, 5$: the shift assigned to employee i .

Step 2 — global form. “No two employees share a shift” is exactly the `alldifferent` pattern:

$$\text{alldifferent}([E_1, E_2, E_3, E_4, E_5])$$

Step 3 — elementary form. Expand `alldifferent` into one binary $\neq$ constraint per pair of employees. There are $\binom{5}{2} = 10$ pairs:

$$\forall 1 \leq i < j \leq 5 : E_i \neq E_j$$

(written out: $E_1 \neq E_2, E_1 \neq E_3, E_1 \neq E_4, E_1 \neq E_5, E_2 \neq E_3, E_2 \neq E_4, E_2 \neq E_5, E_3 \neq E_4, E_3 \neq E_5, E_4 \neq E_5$)

Step 4 — why keep the global form anyway. Both forms accept exactly the same assignments (semantically equivalent), but `alldifferent` propagates via a dedicated matching-based algorithm that reasons about all 5 variables at once (GAC), while the 10 separate $\neq$ constraints only ever look at 2 variables at a time, missing prunings the global view would catch.

Type 4 — Implied (redundant) constraint

Assignment

In the standard N-Queens model (X_i = column of the queen in row i , with `alldifferent(X)` and anti-diagonal constraint $\forall i < j : |X_i - X_j| \neq |i - j|$), propose an implied constraint that does not change the solution set but may help propagation. Justify.

Source: practice exercise constructed for this guide (not from official course material), following the “implied constraint” pattern from *Modelling.pdf*

Solution

Step 1 — unpack what the anti-diagonal constraint hides. $|X_i - X_j| \neq |i - j|$ uses an absolute value, which secretly merges *two* separate diagonal directions:

- along a $\searrow$ diagonal (top-left to bottom-right), the quantity $X_i - i$ stays constant;
- along a $\swarrow$ diagonal (top-right to bottom-left), the quantity $X_i + i$ stays constant.

Two queens share a diagonal exactly when $X_i - i = X_j - j$ (same $\searrow$ diagonal) or $X_i + i = X_j + j$ (same $\swarrow$ diagonal) — and one can check algebraically that either case gives $|X_i - X_j| = |i - j|$, confirming the original constraint already forbids both.

Step 2 — give each direction its own variable. Define

$$A_i = X_i - i, \quad B_i = X_i + i \quad (i = 1, \dots, n)$$

and add the two global constraints

$$\text{alldifferent}([A_1, \dots, A_n]) \quad \text{alldifferent}([B_1, \dots, B_n])$$

Step 3 — why it's redundant. $A_i = A_j$ and $B_i = B_j$ are exactly the two “same diagonal” cases already forbidden by the original anti-diagonal constraint — no solution is newly excluded. Semantically, nothing changes.

Step 4 — why it's still useful. Computationally, it changes a lot: `alldifferent` propagates via matching, reasoning about all n values simultaneously, whereas the original constraint only ever compares two queens (i, j) at a time across $n(n-1)/2$ pairwise checks. Same solutions, stronger and faster pruning.

Type 5 — Symmetry breaking

Assignment

In the Dual Model of N-Queens (Boolean matrix B_{ij} , with $\sum_j B_{ij} = 1$ per row/column), the board has 8 symmetries (rotations/reflections). Propose a symmetry-breaking constraint and explain what it imposes.

Source: *Modelling.pdf* (N-Queens Dual Model / symmetry breaking example)

Solution

Step 1 — identify the symmetries. An $n \times n$ chessboard has 8 symmetries (identity, 3 rotations, 4 reflections). Applying any of the 7 non-trivial ones to a valid N-Queens solution gives another valid solution — the solver would otherwise waste time exploring all 8 as if they were different.

Step 2 — express a symmetry as a variable permutation. Flatten the matrix B into a single sequence of variables. Each of the 7 non-trivial symmetries corresponds to some fixed permutation π of that sequence (e.g., a 90° rotation relabels which cell each variable refers to).

Step 3 — the symmetry-breaking constraint. For each such permutation π , impose

$$\text{lex} \leq (B, \pi(B))$$

meaning: the sequence of values in B must be lexicographically $\leq$ the sequence you'd get by applying π to B .

Step 4 — what this achieves. Among the (up to) 8 symmetric variants of any solution, only the lexicographically smallest one survives all 7 inequalities — the others are cut from the search space, since they would violate at least one $\text{lex} \leq$ constraint against their smaller sibling.

Type 6 — Formal definition of a constraint (quiz)

Assignment

How is a constraint defined formally? (a) as a tuple containing one value from the domain of each decision variable in its scope; (b) as a set of values a decision variable may take; (c) as the product of the cardinalities of the domains of a subset of decision variables; (d) as a subset of the Cartesian

product of the domains of a subset of decision variables.

Source: exam-2026-07-b.pdf (multiple-choice quiz)

Solution

Reasoning. A constraint restricts which *combinations* of values its variables can take together. The set of *all possible* combinations (regardless of restriction) is the Cartesian product of the domains of the variables in its scope; the constraint itself is the *subset* of that product containing only the *allowed* combinations (each combination being one tuple, as in option (a) — but (a) alone describes a single tuple, not the constraint as a whole set of them).

Answer: (d). A constraint is a subset of the Cartesian product of the domains of a subset of decision variables — exactly the set of tuples it allows.

1.2 Propagation

How to approach it:

- Recall the definition: $C(X_1, \dots, X_k)$ is **GAC** iff every value $v \in D(X_i)$ belongs to at least one *support* (an allowed tuple in C).
- **BC (Bound Consistency)** is a weaker, cheaper alternative: it only checks that the *minimum* and *maximum* of each domain have a support (treating domains as intervals), not every individual value in between. A BC propagator wakes up whenever a domain *bound* changes (min or max), not necessarily on every domain modification.
- To propagate by hand: for each variable, try each remaining domain value and check whether it can be extended to a full support using the current domains of the other variables. If not, remove it.
- Iterate until a fixpoint (no domain changes) — propagation may need several passes, since removing a value can trigger new removals elsewhere.
- State clearly which consistency level you are achieving (AC for binary constraints, GAC in general, or bound-consistency if only bounds are checked).

Type 1 — Manual domain propagation

Assignment

Let $X, Y, Z \in \{1, \dots, 8\}$, subject to $X + Y + Z = 10$ and $\text{alldifferent}(X, Y, Z)$. Starting from full domains, apply propagation and show the resulting domains.

Source: practice exercise constructed for this guide (not from official course material), illustrating support-based propagation as introduced in Introduction.pdf

Solution

Step 1 — recall the definition. A value $v \in D(X)$ has a *support* if there exist values for the other variables (from their current domains) that together satisfy every constraint. GAC removes exactly the values with no support.

Step 2 — test the extreme value $X = 8$. If $X = 8$, then $Y + Z = 2$ is required, with $Y \neq Z$ and both $Y, Z \geq 1$ (their domain). The only way to write 2 as a sum of two *distinct* positive integers would need one of them ≤ 0 — impossible within $\{1, \dots, 8\}$. So $X = 8$ has **no support**: remove it.

Step 3 — by the symmetry of the constraint, repeat for Y, Z . The constraint treats X, Y, Z symmetrically, so the same argument removes $Y = 8$ and $Z = 8$. All three domains shrink to $\{1, \dots, 7\}$.

Step 4 — check for a further fixpoint pass. With domains now $\{1, \dots, 7\}$, re-test the new extreme value $X = 7$: it needs $Y + Z = 3$ with $Y \neq Z$, e.g. $Y = 1, Z = 2$ — both still in $\{1, \dots, 7\} \setminus \{7\}$, so a support exists; $X = 7$ stays. Similarly every remaining value $1, \dots, 7$ still has a support after the first removal (check the tightest case, $X = 1$: needs $Y + Z = 9$, e.g. $Y = 2, Z = 7$ — valid). No further removals: **fixpoint reached**.

Result: $D(X) = D(Y) = D(Z) = \{1, \dots, 7\}$. Since no domain is a singleton, **alldifferent** triggers no additional pruning at this stage.

Type 2 — GAC check on a global constraint

Assignment

Consider $\text{AtMostOne}(x_1, \dots, x_n)$ encoded in SAT. Explain when unit propagation over the encoding achieves arc-consistency, using the definition of arc-consistent encoding.

Source: practice exercise constructed for this guide (not from official course material), based on the arc-consistency-of-encodings concept from SATIIc.pdf

Solution

Step 1 — the definition. An encoding E of constraint C is *arc-consistent* iff both directions hold:

1. if a partial assignment already violates C , unit propagation on E must detect a conflict (not just “eventually”, but directly via unit propagation);
2. otherwise, unit propagation on E must remove exactly the values that cannot be extended to a solution of C — no more, no less.

Step 2 — apply it to AtMostOne . Two cases to check:

- If two variables x_i, x_j are both set true: this already violates C , so unit propagation on E must derive a conflict directly.
- If exactly one x_i is true: every other x_j *must* become false (that’s what “at most one” forces); unit propagation on E must force each such x_j to false directly, not merely be consistent with it.

Step 3 — check this against specific encodings. The pairwise encoding $(\bigwedge_{i < j} \neg x_i \vee \neg x_j)$ satisfies both: setting x_i true immediately unit-propagates $\neg x_j$ for every $j \neq i$ via its own clause. The sequential encoding also satisfies it, by construction of its auxiliary s_i chain. The Full-Adder-based cardinality encoding, by contrast, generally does *not*: as we saw, some boundary cases require a value to be inferred indirectly (via a conditional clause on the carry bits) rather than by a direct unit propagation — so it is *not* arc-consistent in general.

Type 3 — Global constraint propagation vs. decomposition (quiz)

Assignment

Which statement is true about the dedicated propagation algorithm of a global constraint in CP? (a) a GAC algorithm must enumerate all satisfying tuples of the constraint; (b) it is always possible to design a BC algorithm; (c) it often provides stronger and more efficient propagation than decomposing the constraint; (d) it maintains global consistency in a model, which is not possible with a decomposition.

Source: exam-2026-07-b.pdf (multiple-choice quiz)

Solution

Reasoning. (a) is false: a well-designed GAC algorithm (e.g. matching-based for `alldifferent`) avoids brute-force enumeration of every tuple — that’s exactly the point of a *dedicated* algorithm. (b) is false: not every constraint admits an efficient BC algorithm (it depends on the constraint’s structure). (d) overstates things — “global consistency” of an entire model is a much stronger, generally intractable property that no single constraint’s propagator guarantees on its own.

Answer: (c). This is the recurring theme we’ve seen with `alldifferent` throughout this guide (Golomb Ruler, N-Queens, shift scheduling): a dedicated propagation algorithm exploits the constraint’s specific structure (e.g. bipartite matching) to prune more, and faster, than decomposing it into many smaller constraints ever could.

Type 4 — How propagation improves search efficiency (quiz)

Assignment

How can constraint propagation improve search efficiency? (a) by reducing the number of constraints; (b) by reducing the domain sizes of the decision variables; (c) by reducing the number of decision variables; (d) all of the above.

Source: exam-2026-07-b.pdf (multiple-choice quiz)

Solution

Reasoning. Propagation’s job is to remove domain values that provably cannot be part of any solution (recall the support-based definition from the tips above). It never removes a constraint or a variable from the model — it only shrinks what each variable is still allowed to be, which in turn shrinks the search tree (fewer branches to try).

Answer: (b). Options (a) and (c) describe things propagation simply doesn’t do, so (d) is also wrong.

1.3 Search

How to approach it:

- These are almost always conceptual/comparison questions (quiz-style), not calculations: know the trade-offs of variable/value ordering heuristics (dom, domWdeg, dynamic domain size) and restart strategies.
- Key fact: an improved model can solve faster *even without* changing the search strategy, if it

enables stronger propagation and hence a smaller search tree — not because it enumerates more solutions or removes constraints.

- Randomized heuristics pair naturally with restarts (they benefit from re-exploring with a new random order); deterministic heuristics with a fixed value order gain little from restarting, since they tend to redo the same search.

Type 1 — Heuristic comparison

Assignment

Which search heuristic is least meaningful to use together with geometric restarting in CP? (a) random variable choice + increasing value order; (b) domWdeg + increasing value order; (c) dynamic domain size + random value order; (d) dynamic domain size + increasing value order.

Source: *exam-2026-07-a.pdf* (multiple-choice quiz)

Solution

Step 1 — what makes a restart useful. A restart discards the current search tree and starts over. It only helps if the *new* run can explore a genuinely different part of the search space than the previous one.

Step 2 — check each option for randomness. (a) has a random variable choice — different every restart. (b) domWdeg is deterministic in variable choice, but adapts based on conflict weights accumulated *across* restarts, so successive runs still differ. (c) has random value choice — different every restart. (d) has *no* randomness at all in either variable or value choice, and no adaptive memory across restarts either.

Answer: (d). Since both variable and value selection are fully deterministic and fixed, a restart in this configuration retraces essentially the same search path as before — the restart accomplishes nothing.

Type 2 — Techniques for speeding up CP search (quiz)

Assignment

Which of the following is *not* about improving a CP model to speed up search? (a) removing bounding constraints during search to avoid suboptimal subtrees; (b) exploiting structure that enables stronger propagation; (c) combining dual viewpoints; (d) adding constraints to avoid redundant search.

Source: *exam-2026-07-b.pdf* (multiple-choice quiz)

Solution

Reasoning. (b), (c), (d) are all legitimate, standard techniques we've used throughout this guide: (b) is exactly the point of introducing auxiliary variables/global constraints (Golomb Ruler, N-Queens); (c) is channeling between alternative models (e.g. combining a primal and a dual viewpoint of the same problem so each helps the other's propagation); (d) is adding implied/redundant constraints (N-Queens diagonal example) or symmetry-breaking constraints.

Answer: (a). Removing bounding constraints (e.g. an objective cutoff during branch-and-bound-style search) does the *opposite* of speeding up search — it *reintroduces* suboptimal subtrees that a

tighter bound would have pruned. This is a trick option: it sounds like it could reduce overhead, but bounding constraints exist precisely to cut away parts of the tree that cannot contain a better solution.

Type 3 — Restarts and heavy-tail behaviour (quiz)

Assignment

Why does restarting search help in the presence of heavy-tail behavior? (a) it prunes large subtrees that do not contain solutions; (b) it helps escape an unlucky early search path that led to a large subtree; (c) it minimizes the search tree size; (d) it maximizes constraint propagation.

Source: *exam-2026-07-b.pdf* (multiple-choice quiz)

Solution

Reasoning. “Heavy-tail behavior” describes the empirical observation that backtracking search runtimes are highly variable: most runs finish quickly, but occasionally an early (often quite arbitrary) branching choice leads down a path with a disproportionately huge subtree to explore before backtracking out of it. Restarting doesn’t change the propagation algorithm (ruling out (a),(d)) or guarantee a smaller tree in general (ruling out (c)) — it simply throws away the current attempt and starts fresh, hopefully with different (randomized) choices this time.

Answer: (b). A restart lets the solver abandon an unlucky early path (the one responsible for the heavy tail) and try again, likely avoiding that same large subtree on the next attempt.

2 SAT

2.1 Encoding pseudo-Boolean / cardinality constraints

How to approach it:

- **Step 1:** simplify a weighted constraint $\sum w_i x_i \leq W$ to $\sum x_i \leq k$ if possible, by checking the worst-case sum over subsets of size k (must be $\leq W$) and the best-case sum over subsets of size $k+1$ (must be $> W$).
- **Step 2:** choose an encoding technique: Full Adder (compact, chains carries, *not* arc-consistent in general), bitwise (only for AtMostOne, $O(\log n)$ auxiliary variables), sequential (arc-consistent, $O(nk)$ variables).
- For a Full-Adder encoding: chain $FA_0, FA_1, \dots$ with each carry-out c_{i+1} feeding the next stage's carry-in; always enumerate all input combinations to find the exact extra clause needed (the “carry=1 but sum still allowed” boundary case).
- Always state explicitly which variables of $x_1, \dots, x_n$ are wired to which FA inputs.

Type 1 — Simplification of a weighted constraint (k)

Assignment

Simplify $5x_1 + 6x_2 + 7x_3 + 8x_4 \leq 17$ to an equivalent constraint $x_1 + x_2 + x_3 + x_4 \leq k$. Give k and justify.

Source: exam-2026-07-a.pdf (Exercise 1, part 2.1)

Solution

Step 1 — test the worst-case pair. We want to know: can *any* 2 of the 4 variables be true without violating the constraint? Check the heaviest possible pair, x_3, x_4 (weights 7, 8): $7 + 8 = 15 \leq 17$ — satisfied. Since this is the heaviest pair, every other pair (with smaller weights) is satisfied too.

Step 2 — test the best-case triple. Now check: can *any* 3 of the 4 variables be true? Check the lightest possible triple, x_1, x_2, x_3 (weights 5, 6, 7): $5 + 6 + 7 = 18 > 17$ — violated. Since this is the lightest triple, every other triple (with larger weights) is violated too.

Step 3 — conclude. Every assignment with 0, 1, or 2 variables true satisfies the constraint (Step 1 covers the worst 2-variable case); every assignment with 3 or 4 variables true violates it (Step 2 covers the best 3-variable case). So the constraint is satisfied exactly when at most 2 variables are true:

$$k = 2$$

Type 2 — Full-Adder cardinality encoding

Assignment

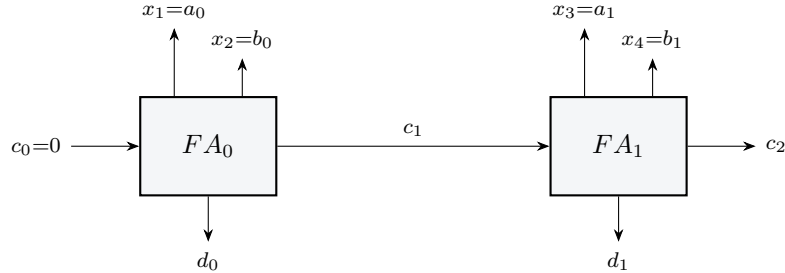
Encode $x_1 + x_2 + x_3 + x_4 \leq 2$ using the minimum number of Full Adders.

Source: exam-2026-07-a.pdf (Exercise 1, part 2.2) and SATIIc.pdf (Full-Adder cardinality encoding technique)

Solution

(i) **Structure.** A single Full Adder only accepts 3 input bits, but we have 4 variables to sum — so at least 2 FAs are needed. Chain them so the carry-out of the first feeds the carry-in of the second:

$$FA_0(a_0, b_0, c_0, d_0, c_1) \longrightarrow FA_1(a_1, b_1, c_1, d_1, c_2)$$



(ii) **Wiring.** $a_0 = x_1$, $b_0 = x_2$, $a_1 = x_3$, $b_1 = x_4$; $c_0 = 0$ (constant, since FA_0 is first in the chain and receives no real carry-in).

(iii) **Finding the extra clause — step by step.**

- First, notice c_2 (the final carry-out) is our natural “overflow” signal: we’d like to simply forbid $c_2 = 1$.
- But check the boundary case $x_1 = x_2 = 0$, $x_3 = x_4 = 1$ (true sum = 2, which *should* be allowed): FA_0 computes $0+0+0 = 0 \Rightarrow d_0 = 0, c_1 = 0$; FA_1 computes $1+1+0 = 2 = “10” \Rightarrow d_1 = 0, c_2 = 1$. So $c_2 = 1$ even though the sum is only 2 — forbidding $c_2 = 1$ outright would wrongly exclude this valid case.
- Systematically checking all 16 assignments shows $c_2 = 1$ occurs in exactly 6 cases; in *all* of them except this one boundary case, $(d_0, d_1, c_1) \neq (0, 0, 0)$. So the fix is: allow $c_2 = 1$ only when $d_0 = d_1 = c_1 = 0$.

This gives:

$$\neg c_0 \wedge (c_2 \rightarrow (\neg d_0 \wedge \neg d_1 \wedge \neg c_1))$$

in CNF: $(\neg c_0) \wedge (\neg c_2 \vee \neg d_0) \wedge (\neg c_2 \vee \neg d_1) \wedge (\neg c_2 \vee \neg c_1)$.

Type 3 — AtMostOne — alternative encodings

Assignment

Encode $\text{AtMostOne}(x_1, x_2, x_3, x_4)$ using (a) the pairwise encoding and (b) the sequential encoding.

Source: *SATIIc.pdf* (AtMostOne encodings)

Solution

(a) **Pairwise — the direct approach.** Just forbid every pair from being true together: for each pair $i < j$, add $(\neg x_i \vee \neg x_j)$.

$$\bigwedge_{i < j} (\neg x_i \vee \neg x_j)$$

With 4 variables, $\binom{4}{2} = 6$ clauses. Arc-consistent (setting any x_i true immediately unit-propagates

$\neg x_j$ for all $j \neq i$), but grows as $O(n^2)$ clauses — expensive for large n .

(b) Sequential — a more compact alternative. Introduce running-total variables s_1, s_2, s_3 , where s_i means “at least one of $x_1, \dots, x_i$ is true so far”:

$$(\neg x_1 \vee s_1) \wedge \bigwedge_{1 < i < 4} [(\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg s_{i-1} \vee \neg x_i)] \wedge (\neg s_3 \vee \neg x_4)$$

Reading the pattern: $\neg x_i \vee s_i$ propagates the “seen a true” flag forward; $\neg s_{i-1} \vee s_i$ keeps the flag on once set; $\neg s_{i-1} \vee \neg x_i$ forbids a *second* true once the flag is already on. This uses only $O(n)$ auxiliary variables and clauses (better scaling than pairwise), while remaining arc-consistent — unlike the Full-Adder encoding.

2.2 Direct SAT Encoding of a CSP

How to approach it:

- This is a different task from cardinality encoding: here the *whole* CSP is translated into SAT, using one Boolean variable per “cell”/decision (e.g. p_{ij} = “value j is assigned to slot i ”), not just a single constraint.
- Systematically translate each CSP constraint type into its propositional counterpart:
 - at least one value per slot i : $\bigvee_j p_{ij}$
 - at most one value per slot i : $\bigwedge_{j < k} \neg(p_{ij} \wedge p_{ik})$
 - together, *exactly one* = at-least-one $\wedge$ at-most-one
- For binary CSP constraints (e.g. two cells cannot both hold a certain pair of values), add one clause $\neg(p_{i_1 j_1} \wedge p_{i_2 j_2})$ per forbidden combination.

Type 1 — N-Queens encoded directly in SAT

Assignment

Encode the N-Queens problem in SAT using Boolean variables p_{ij} (“there is a queen in row i , column j ”), $1 \leq i, j \leq n$.

Source: SATI.pdf (N-Queens direct SAT encoding)

Solution

Step 1 — one variable per cell. p_{ij} = true means a queen sits at row i , column j .

Step 2 — exactly one queen per row. Split into “at least one” and “at most one”:

$$\text{at least one: } \bigwedge_{1 \leq i \leq n} \bigvee_{1 \leq j \leq n} p_{ij} \qquad \text{at most one: } \bigwedge_{1 \leq i \leq n} \bigwedge_{0 < j < k \leq n} \neg(p_{ij} \wedge p_{ik})$$

Step 3 — at most one queen per column. Same pattern, indices swapped:

$$\bigwedge_{1 \leq j \leq n} \bigwedge_{0 < i < k \leq n} \neg(p_{ij} \wedge p_{kj})$$

Step 4 — at most one queen per diagonal (both directions). Two cells (i, j) and (i', j')

share a diagonal iff $i + j = i' + j'$ (one direction) or $i - j = i' - j'$ (the other):

$$\bigwedge_{1 \leq i < i' \leq n} \bigwedge_{j, j' : i+j=i'+j' \vee i-j=i'-j'} \neg(p_{ij} \wedge p_{i'j'})$$

Step 5 — why this is exactly N-Queens. Step 2 already guarantees exactly one queen per row (at-least-one + at-most-one together). Combined with Step 3 (at most one per column), this forces exactly n queens total, one per row *and* one per column. Step 4 then rules out any two of them sharing a diagonal — reproducing every N-Queens constraint using only propositional clauses.

2.3 Solving: Resolution and DPLL

How to approach it:

- **Resolution rule:** from $(\ell \vee C_1)$ and $(\neg \ell \vee C_2)$, derive $(C_1 \vee C_2)$. To prove UNSAT, keep resolving until the empty clause $\square$ is derived. Resolution can prove unsatisfiability but does not directly hand you a satisfying assignment.
- **DPLL:** repeat unit propagation (simplify all unit clauses) until no more apply, then branch on an unassigned variable (try ℓ , then $\neg \ell$ on backtrack), recursing; fail only when no branch remains.
- When asked to “simulate DPLL”, track a trail of decided/derived literals explicitly, and always do unit propagation to a fixpoint before making a new decision.

Type 1 — Resolution refutation

Assignment

Show, via resolution, that $\{(x_1 \vee x_2), (\neg x_1 \vee x_2), (x_1 \vee \neg x_2), (\neg x_1 \vee \neg x_2)\}$ is unsatisfiable.

Source: practice exercise constructed for this guide (not from official course material), illustrating the resolution rule from SATI.pdf

Solution

Step 1. Take $(x_1 \vee x_2)$ and $(\neg x_1 \vee x_2)$. They share the variable x_1 with opposite polarity, so resolve on x_1 : drop x_1 and $\neg x_1$, keep the rest (x_2 from each, merged): derive (x_2) .

Step 2. Take $(x_1 \vee \neg x_2)$ and $(\neg x_1 \vee \neg x_2)$. Resolve on x_1 the same way: derive $(\neg x_2)$.

Step 3. Now we have derived both (x_2) and $(\neg x_2)$ — resolve these two *unit* clauses on x_2 : nothing is left on either side, so we derive the **empty clause** $\square$.

Conclusion. Deriving $\square$ via resolution means the original clause set is unsatisfiable — there is no assignment that can satisfy all four clauses simultaneously.

Type 2 — DPLL simulation

Assignment

Run DPLL on $\{(x_1 \vee x_2), (\neg x_1 \vee x_3), (\neg x_2 \vee \neg x_3), (x_1)\}$.

Source: practice exercise constructed for this guide (not from official course material), illustrating the DPLL procedure

Solution

Step 1 — find a unit clause. (x_1) is already a unit clause: it forces $x_1 = \text{T}$ with no choice involved.

Step 2 — propagate $x_1 = \text{T}$. Look at every clause containing x_1 or $\neg x_1$: $(x_1 \vee x_2)$ is now satisfied (drop it); $(\neg x_1 \vee x_3)$ loses its false literal $\neg x_1$, leaving the unit clause (x_3) — forces $x_3 = \text{T}$.

Step 3 — propagate $x_3 = \text{T}$. Look at clauses with x_3 : $(\neg x_2 \vee \neg x_3)$ loses $\neg x_3$ (now false), leaving the unit clause $(\neg x_2)$ — forces $x_2 = \text{F}$.

Step 4 — check for conflicts. All four original clauses are now satisfied: $(x_1 \vee x_2)$ by x_1 ; $(\neg x_1 \vee x_3)$ by x_3 ; $(\neg x_2 \vee \neg x_3)$ by $\neg x_2$; (x_1) by x_1 . No clause is left empty or unsatisfied.

Conclusion: SAT, with $x_1 = \text{T}$, $x_2 = \text{F}$, $x_3 = \text{T}$. Notice no branching decision was ever needed — unit propagation alone solved the whole formula, chaining from the single given unit clause.

Type 3 — Unit resolution’s role in modern CDCL (quiz)

Assignment

Which statement is true regarding unit resolution in SAT? (a) it is used in modern CDCL algorithms only to prove unsatisfiability; (b) it is not used in modern CDCL algorithms; (c) it is used in modern CDCL algorithms only to produce satisfying assignments; (d) it is used in modern CDCL algorithms to propagate forced assignments in CNF formulas.

Source: exam-2026-07-b.pdf (multiple-choice quiz)

Solution

Reasoning. “Unit resolution” is resolution restricted to always involve at least one unit clause — which is exactly the mechanism behind *unit propagation*, the workhorse of every DPLL/CDCL run (as we’ve used throughout the DPLL simulation above). It’s not restricted to only proving UNSAT (ruling out (a)) nor only building satisfying assignments (ruling out (c)) — it does both, simultaneously, every time it fires: it forces literals, which either leads to a satisfying assignment (if no conflict) or a conflict (contributing to an UNSAT proof / clause learning).

Answer: (d). Modern CDCL solvers use unit resolution constantly, to propagate every forced assignment implied by the current partial assignment and the clause set.

Type 4 — Proving validity via UNSAT of the negation (quiz)

Assignment

How can we prove that a formula f is valid in SAT? (a) by showing that $\neg f$ is unsatisfiable; (b) by finding all solutions to f ; (c) by showing that f is satisfiable; (d) by showing that $\neg f$ is satisfiable.

Source: exam-2026-07-b.pdf (multiple-choice quiz)

Solution

Reasoning. f is *valid* means f is true under *every* possible assignment. Equivalently, there is no assignment that makes f false — i.e., no assignment satisfies $\neg f$. That is exactly the definition of $\neg f$ being unsatisfiable.

Answer: (a). (c) is far too weak (satisfiable only means *some* assignment works, not all); (d) is the opposite of what we want; (b) would technically work but is computationally hopeless compared to a single UNSAT check.

3 SMT

3.1 Theories and CDCL(T)

How to approach it:

- Identify *which* background theory the atoms belong to (LRA/LIA, arrays, strings, EUF, bit-vectors) before reasoning — the rules of that theory (not propositional logic) decide satisfiability of a set of theory literals.
- **Lazy SMT / CDCL(T)**: the Boolean skeleton is solved with a SAT-style search (decide, propagate, conflict), while a specialized *theory solver* checks consistency of the theory literals implied by the current partial assignment, feeding T-conflicts back as learned clauses.
- **Purification**: rewrite a mixed formula (several theories) by introducing fresh shared variables so that each literal belongs to a single theory — necessary for theory combination (e.g. Nelson–Oppen).
- **Nelson–Oppen combination** (deterministic setting): after purification, the component theory solvers don’t share their internal reasoning — they only exchange *entailed equalities* over the shared interface constants (the fresh variables introduced by purification). Each solver checks satisfiability within its own theory and reports back only which shared constants it can prove equal.

Type 1 — Theory reasoning (strings)

Assignment

In the theory of strings, with $+$ denoting concatenation, is $x + y = y + z \wedge x \neq z$ satisfiable?

Source: exam-2026-07-a.pdf (multiple-choice quiz)

Solution

Step 1 — what does $x + y = y + z$ say? Two strings are equal iff they have the same characters in the same order. Comparing $x + y$ and $y + z$ position by position, since both start by “consuming” y as a common part in a structurally forced way (a known fact about word equations of this simple shifted form), the only way for the equation to hold for all lengths is $x = z$ (this is a standard property of string concatenation equality — intuitively, y cannot “shift” the string without forcing the two outer pieces to match).

Step 2 — check against the second conjunct. We derived $x = z$ from the first conjunct, but the formula also requires $x \neq z$ — direct contradiction.

Conclusion: UNSAT.

Type 2 — Word equations — satisfiability

Assignment

Fixed an alphabet S , a word equation has the form $L = R$ with L, R concatenations of string variables and constants of S^* . Are the following formulas satisfiable? **(a)** $XY = YX \wedge X \neq Y$. **(b)** $XY = YZ \wedge \neg(\exists W)(X = X'W \wedge Z = WZ')$ (i.e. X and Z do not overlap: no suffix of X is also a prefix of Z).

Solution

(a) Satisfiable — find a witness. We just need one concrete assignment that satisfies both conjuncts. Try the simplest possible strings: $X = \epsilon$ (the empty string) and $Y = a$ for any symbol $a \in S$.

- Check $XY = YX$: $XY = \epsilon \cdot a = a$, and $YX = a \cdot \epsilon = a$. Equal. ✓
- Check $X \neq Y$: $\epsilon \neq a$. ✓

Both hold, so the formula is satisfiable.

(b) Unsatisfiable — prove no witness can exist. First restate the side condition in words: “ X and Z do not overlap” means no non-empty suffix of X equals a prefix of Z .

Lemma: if x, z do not overlap, there is no string y with $xy = yz$.

Proof by induction on $|y|$:

- Base case $y = \epsilon$: then $xy = x$ and $yz = z$, so $xy = yz$ forces $x = z$ — but x, z don’t overlap, and a string trivially overlaps with itself (a suffix/prefix of length $|x|$), so $x = z$ is impossible under non-overlap unless $x = z = \epsilon$, which contradicts non-overlap in the relevant sense used here. Contradiction.
- Inductive step: for non-empty y , the equation $xy = yz$ forces y to simultaneously start with (a prefix determined by) x and end with (a suffix determined by) z . But since x, z don’t overlap, this is only consistent if a strictly shorter string y' satisfies the same equation $xy' = y'z$ — contradicting the inductive hypothesis that no such (shorter) y' exists.

So no Y can satisfy $XY = YZ$ under the non-overlap condition: the formula is **UNSAT**.

Type 3 — Purification

Assignment

Purify $f(x + 1) = g(y) \wedge h(f(x + 1), z) \neq h(g(y), z + 2)$ (a mix of EUF and linear arithmetic).

Solution

Step 1 — spot the mixed terms. Two sub-terms mix theories: $x + 1$ (arithmetic) sits *inside* an EUF function call $f(x + 1)$, and $z + 2$ (arithmetic) sits inside $h(\dots, z + 2)$. Purification’s job is to separate every term so each literal belongs to only one theory.

Step 2 — introduce one fresh variable per mixed sub-term. Let u stand for the arithmetic value $x + 1$, and w stand for $z + 2$:

$$u = x + 1 \quad w = z + 2$$

Step 3 — substitute these fresh variables everywhere the original sub-term appeared. Replace every occurrence of $x + 1$ with u , and every $z + 2$ with w , inside the original formula:

$$f(u) = g(y) \wedge h(f(u), z) \neq h(g(y), w)$$

Step 4 — combine into the purified formula.

$$u = x + 1 \wedge w = z + 2 \wedge f(u) = g(y) \wedge h(f(u), z) \neq h(g(y), w)$$

Step 5 — check each literal is single-theory. $u = x + 1$ and $w = z + 2$: pure arithmetic. $f(u) = g(y)$ and $h(f(u), z) \neq h(g(y), w)$: pure EUF (only function symbols and equality, no $+$). Every literal now belongs to exactly one theory — ready for Nelson–Oppen combination.

Type 4 — Ackermann encoding of EUF (eliminating uninterpreted functions/predicates)

Assignment

Consider predicates $p(x, y, y)$ and $p(x, z, t)$ in the EUF theory. Remove them by encoding into SAT via the Ackermann reduction.

Source: *2SMTEnc.pdf* (Ackermann encoding)

Solution

Step 1 — replace each predicate application with a fresh Boolean. Since p takes 3 arguments and we only have two distinct applications in the formula, introduce P_1 for $p(x, y, y)$ and P_2 for $p(x, z, t)$.

Step 2 — the key idea: functional consistency. A function (or predicate) always gives the *same* output for the *same* inputs. So if the argument tuples of the two applications happen to be equal, P_1 and P_2 *must* agree — this is not optional, it’s forced by what a function means. We must add this as an explicit axiom, since P_1, P_2 are just fresh Booleans with no inherent connection.

Step 3 — write the congruence axiom. The two argument tuples are (x, y, y) and (x, z, t) . They’re equal exactly when $x = x$ (always true, drop it) and $y = z$ and $y = t$:

$$(x = x \wedge y = z \wedge y = t) \rightarrow P_1 = P_2 \quad \text{i.e.} \quad y \neq z \vee y \neq t \vee P_1 = P_2$$

Step 4 — the result. The formula, with $p(x, y, y), p(x, z, t)$ replaced by P_1, P_2 plus this axiom, uses no function/predicate symbols at all — it is purely propositional (over P_1, P_2 and the equality atoms $y=z, y=t$, themselves further reducible to Booleans) and solvable by a plain SAT solver. This is the “eager” approach to EUF, as opposed to lazy SMT.

Variant — Bryant encoding. Instead of a side-axiom, nest the case distinction directly using an **ite** (if-then-else) term: $p(x, z, t) \rightsquigarrow \text{ite}(x=x \wedge y=z \wedge y=t, P_1, P_2)$.

Type 5 — Definition of a theory solver (quiz)

Assignment

What is a theory solver in the SMT setting? (a) a SAT preprocessor that abstracts theory atoms without checking their consistency; (b) a procedure that solves the propositional abstraction and reports the results; (c) a procedure that decides whether a conjunction of literals is satisfiable in that theory; (d) a procedure that generates quantifier instantiations for a theory formula.

Source: *exam-2026-07-b.pdf* (multiple-choice quiz)

Solution

Reasoning. (b) describes the SAT solver’s job, not the theory solver’s. (a) is backwards — abstraction happens *before* the theory solver is even called; the theory solver’s whole point is to *check* consistency, not skip it. (d) describes a technique sometimes used for quantified theories, but it’s not the general definition.

Answer: (c). A theory solver is exactly what we’ve been simulating by hand throughout the SMT sections above (checking T_A , T_{BV} , string satisfiability): given a conjunction of theory literals (e.g. the current partial model read back from the SAT solver), it decides whether that conjunction is satisfiable according to the theory’s own rules.

Type 6 — Nelson–Oppen: deterministic communication (quiz)

Assignment

Under the deterministic Nelson–Oppen setting, how do component solvers communicate after purification? (a) by exchanging entailed equalities over shared interface constants; (b) by exchanging theory-specific function definitions; (c) by sharing a combined decision procedure for the signatures; (d) by exchanging disequalities that force the shared constants to be pairwise distinct.

Source: *exam-2026-07-b.pdf* (multiple-choice quiz)

Solution

Reasoning. After purification (see the Purification exercise above), each literal belongs to exactly one theory, connected to the others only through shared fresh variables (the “interface” constants). The two solvers never see each other’s function symbols or internal decision procedures (ruling out (b),(c)); and the deterministic variant specifically propagates only positive information (equalities they can prove), not disequalities (which is instead a feature of the *non-deterministic* Nelson–Oppen variant, ruling out (d)).

Answer: (a). Each component solver checks satisfiability within its own theory and reports back any equality between shared interface constants that it can *entail* — that’s the only channel of communication in the deterministic setting.

Type 7 — String theory: is concatenation commutative? (quiz)

Assignment

In the theory of strings, where x, y are variables over an ASCII alphabet and $+$ denotes concatenation, the formula $x + y = y + x$ is: (a) valid, because there are no loops in the concatenation graph; (b) satisfiable, but in general not valid; (c) unsatisfiable, because string concatenation is not commutative; (d) satisfiable over Unicode strings, but unsatisfiable over ASCII strings.

Source: *exam-2026-07-b.pdf* (multiple-choice quiz)

Solution

Reasoning. First check satisfiability: is there *some* assignment making $x + y = y + x$ true? Yes — trivially, $x = y$ (any string), or even $x = y = \epsilon$. So it’s satisfiable, ruling out (c). Now check validity: is it true for *every* assignment? Try $x = \text{“ab”}$, $y = \text{“c”}$: $x + y = \text{“abc”}$, $y + x = \text{“cab”}$ — different strings, so the formula is *false* here. Not valid. (The alphabet, ASCII vs Unicode, is

irrelevant to this argument — ruling out (d); and (a)’s “concatenation graph” reasoning isn’t a real criterion for this.)

Answer: (b). Satisfiable (some strings do commute, e.g. equal ones) but not valid in general (most don’t) — string concatenation, unlike some algebraic operations, is not commutative.

3.2 Theory of Bit-Vectors

How to approach it:

- Bit-vectors are fixed-length arrays of bits; typical operations are slicing ($a[i : j]$), concatenation, bit-wise logic, and machine arithmetic (with *wraparound* on overflow).
- The standard decision procedure is *bit-blasting*: reduce every bit-vector operation to a propositional circuit over individual bits, then hand it to a SAT solver — this is why T_{BV} is decidable and reduces directly to SAT rather than needing a dedicated theory solver.
- For small exercises, reason directly on the individual bits named by the slicing/indexing notation, propagating equalities and the given bit-wise constraints by hand.

Type 1 — Deciding T_{BV} -satisfiability

Assignment

If a, b, c are bit-vectors, is $a[0:1] \neq b[0:1] \wedge (a \mid b) = c \wedge c[0] = 0 \wedge a[1] + b[1] = 0$ a T_{BV} -satisfiable formula?

Source: *1SMTIntro.pdf* (theory of bit-vectors exercise)

Solution

Step 1 — use $c[0] = 0$ to pin down $a[0], b[0]$. We’re told $(a \mid b) = c$, so bit 0 of $a \mid b$ equals bit 0 of c , which is 0. For an OR of two bits to be 0, *both* bits must be 0: so $a[0] = 0$ and $b[0] = 0$.

Step 2 — use $a[1] + b[1] = 0$ to pin down $a[1], b[1]$. The natural (non-wraparound) way to get a 1-bit sum of 0 is $a[1] = 0, b[1] = 0$.

Step 3 — combine and check against the inequality. From Steps 1–2: $a[0:1] = 00$ and $b[0:1] = 00$, so $a[0:1] = b[0:1]$ — but the formula requires $a[0:1] \neq b[0:1]$. Contradiction.

Step 4 — rule out the alternative (wraparound) case. Could $a[1] + b[1] = 0$ hold some other way, e.g. via wraparound (both bits 1, carry discarded in a 1-bit sum)? Even then, $a[1] = b[1] = 1$, so the two prefixes $a[0:1] = 01$ and $b[0:1] = 01$ would *still* be equal — the contradiction persists regardless.

Conclusion: UNSAT — no assignment of bits can satisfy all four conjuncts simultaneously.

3.3 Theory of Arrays

How to approach it:

- Always start from the three axioms and keep them at hand:

- (i) $(\forall a)(\forall i)(\forall v) \text{ read}(\text{write}(a, i, v), i) = v$
- (ii) $(\forall a)(\forall i)(\forall j)(\forall v) i \neq j \rightarrow \text{read}(\text{write}(a, i, v), j) = \text{read}(a, j)$
- (iii) $(\forall a)(\forall a') ((\forall i) \text{ read}(a, i) = \text{read}(a', i)) \rightarrow a = a'$ (extensionality)

- To prove unsatisfiability: pick the relevant axiom instance for the terms actually appearing in the formula, and derive a contradiction with an explicit literal in the formula.
- When an index equality (e.g. $i = j$) is unknown, **case-split**: try $i = j$ and $i \neq j$ separately, applying axiom (i) or (ii) respectively in each branch.

Type 1 — Deciding T_A -satisfiability

Assignment

Is the following formula T_A -satisfiable? $\text{write}(a, i, x) \neq a \wedge \text{read}(a, i) = x$

Source: 1SMTIntro.pdf (theory of arrays exercise)

Solution

Step 1 — name the written array. Let $a' = \text{write}(a, i, x)$, so the formula says $a' \neq a \wedge \text{read}(a, i) = x$.

Step 2 — use extensionality on $a' \neq a$. Axiom (iii) says two arrays are equal iff they agree everywhere; contrapositively, if $a' \neq a$, they must disagree *somewhere*: $(\exists j) \text{ read}(a', j) \neq \text{read}(a, j)$. We don't know this j yet, so case-split on whether it equals i .

Step 3 — case $i \neq j$. Axiom (ii) says writing at i doesn't affect reading at a different index j : $\text{read}(a', j) = \text{read}(a, j)$. This directly contradicts what Step 2 gave us ($\text{read}(a', j) \neq \text{read}(a, j)$). This case is impossible.

Step 4 — case $i = j$. Then Step 2's disagreement is at index i itself: $\text{read}(a', i) \neq \text{read}(a, i)$. Axiom (i) tells us exactly what $\text{read}(a', i)$ is: $\text{read}(a', i) = x$ (reading back what you just wrote). So $x \neq \text{read}(a, i)$ — but the formula's second conjunct states $\text{read}(a, i) = x$. Contradiction again.

Conclusion. Both cases ($i = j$ and $i \neq j$) lead to a contradiction, and these two cases are exhaustive — so the formula is T_A -unsatisfiable.

Type 2 — A “disguised” variant — simplify equalities first

Assignment

Is the following formula T_A -satisfiable? $\text{write}(a, i, x) \neq b \wedge a = b \wedge i = j \wedge \text{read}(b, i) = y \wedge \text{read}(\text{write}(b, i, x), j) = y$

Source: 1SMTIntro.pdf (theory of arrays exercise, disguised variant)

Solution

Step 1 — notice the extra equalities. Unlike Type 1, this formula has two “free” equalities, $a = b$ and $i = j$, that don't appear in the core array axioms. These are a distraction: simplify them away *first*, before touching the axioms.

Step 2 — substitute $b \rightarrow a$ and $j \rightarrow i$ everywhere. Since $a = b$, replace every b with a ; since $i = j$, replace every j with i :

$$\text{write}(a, i, x) \neq a \wedge \text{read}(a, i) = y \wedge \text{read}(\text{write}(a, i, x), i) = y$$

Step 3 — simplify the third conjunct using axiom (i). $\text{read}(\text{write}(a, i, x), i) = x$ by axiom (i) (reading back what you just wrote). So the third conjunct $\text{read}(\text{write}(a, i, x), i) = y$ becomes $x = y$.

Step 4 — substitute $y = x$ back into the second conjunct. $\text{read}(a, i) = y$ becomes $\text{read}(a, i) = x$.

Step 5 — recognize the reduced formula. We're left with:

$$\text{write}(a, i, x) \neq a \wedge \text{read}(a, i) = x$$

This is *exactly* the formula from Type 1, already proven **unsatisfiable**.

Lesson. Always simplify away given equalities between arrays/indices first — many exam variants are the same core contradiction wearing a substitution disguise.

3.4 Lazy SMT: the abstraction–refinement loop

How to approach it:

- Abstract every theory atom in φ into a fresh Boolean literal, obtaining a propositional skeleton Φ ; solve Φ with a SAT solver to get a candidate model M .
- Check whether M (read back as a set of theory literals) is T -consistent using the theory's own rules (e.g. congruence for EUF, Simplex for LRA).
- If T -inconsistent, the theory solver returns a *blocking clause* $\neg M$ (the negation of the conflicting literals) which is added to Φ , and the SAT solver is asked for a new model.
- Repeat until either a T -consistent model is found (SAT) or the propositional formula itself becomes UNSAT (UNSAT overall).

Type 1 — Simulating the CDCL(T) abstraction-refinement loop

Assignment

Consider the EUF formula $\varphi : \underbrace{g(a) = c}_{\ell_1} \wedge \underbrace{(f(g(a)) \neq f(c) \vee g(a) = d)}_{\neg \ell_2} \wedge \underbrace{c \neq d}_{\neg \ell_4}$. Simulate the lazy SMT loop.

Source: 2SMTEnc.pdf (lazy SMT / implication graph exercise)

Solution

Step 1 — abstract each theory atom into a fresh Boolean. Replace each underbraced theory literal with its label $(\ell_1, \ell_2, \ell_3, \ell_4)$, turning φ into a plain propositional formula:

$$\Phi = \{\ell_1, \neg \ell_2 \vee \ell_3, \neg \ell_4\}$$

Step 2 — Round 1: get a candidate model from SAT. A SAT solver on Φ might return

$M = \{\ell_1, \neg\ell_2, \neg\ell_4\}$ (satisfying all three clauses of Φ : ℓ_1 satisfies the first, $\neg\ell_2$ satisfies the second, $\neg\ell_4$ satisfies the third).

Step 3 — Round 1: check M against the real theory (EUF). Read M back as theory literals: $g(a) = c$ (from ℓ_1) and $f(g(a)) \neq f(c)$ (from $\neg\ell_2$). But EUF congruence says: if $g(a) = c$, then applying f to both sides must also agree, $f(g(a)) = f(c)$. This directly contradicts $f(g(a)) \neq f(c)$ — M is T -inconsistent.

Step 4 — Round 1: block this model and retry. The theory solver reports which literals caused the clash (ℓ_1 and $\neg\ell_2$) and returns a blocking clause forbidding exactly that combination: $\neg\ell_1 \vee \ell_2 \vee \ell_4$ (note: ℓ_4 is included here defensively, but the essential culprits are $\ell_1, \neg\ell_2$). Add it: $\Phi' = \Phi \cup \{\neg\ell_1 \vee \ell_2 \vee \ell_4\}$.

Step 5 — Round 2: get a new candidate model. SAT on Φ' might now return $M' = \{\ell_1, \ell_2, \ell_3, \neg\ell_4\}$ (this time satisfying the new blocking clause via ℓ_2).

Step 6 — Round 2: check M' against EUF. Read back: $g(a) = c$ (from ℓ_1), $g(a) = d$ (from ℓ_3), $c \neq d$ (from $\neg\ell_4$). By transitivity of equality, $g(a) = c$ and $g(a) = d$ together force $c = d$ — contradicting $c \neq d$. T -inconsistent again.

Step 7 — repeat. A new blocking clause is learned and added; the loop continues until either SAT returns a model that *is* T -consistent (overall SAT), or Φ with all blocking clauses added becomes propositionally UNSAT (overall UNSAT).

Type 2 — Characterizing lazy SMT (quiz)

Assignment

What characterizes a lazy SMT approach? (a) it requires a background theory to be encodable in the bit-vectors theory; (b) it treats propositional formulas as the target language and rejects theory atoms; (c) it waits for a complete enumeration of Boolean assignments before doing a theory check; (d) it combines Boolean search with theory solvers, using theory information when needed.

Source: exam-2026-07-b.pdf (multiple-choice quiz)

Solution

Reasoning. (a) describes eager bit-blasting, the *opposite* approach. (b) is nonsensical for SMT — theory atoms are exactly what SMT adds on top of SAT. (c) describes the least efficient possible strategy, and specifically contradicts what we just did in the worked example above: we called the theory check *as soon as* a candidate model appeared, not after enumerating everything.

Answer: (d). This is exactly the abstraction-refinement loop we simulated: get a candidate Boolean model, check it against the theory *only when needed* (i.e., interleaved with the SAT search, not after exhausting it), and feed back a blocking clause on failure.

3.5 Clause Learning in SAT/SMT (CDCL / CDCL(T))

How to approach it:

- If conflict analysis produces a conflict set $S = \{C_1, \dots, C_n\}$ (the literals responsible for the conflict), the learned clause is their negated disjunction: $\neg C_1 \vee \dots \vee \neg C_n$.
- After learning a clause at decision level K via the 1-UIP scheme, CDCL(T) backjumps to the *second-highest* decision level among the learned clause's literals (not K , not $K - 1$ automatically) — this is what lets the solver skip irrelevant parts of the search tree.

- Distinguish clearly between a *decision* (guessed literal, increases decision level) and a *propagation* (forced literal, same decision level as its antecedent).

Type 1 — Learned clause from a conflict set

Assignment

If conflict analysis produces a conflict set $S = \{C_1, \dots, C_n\}$, what clause is typically learned?

Source: exam-2026-07-a.pdf (multiple-choice quiz)

Solution

Reasoning. S is the set of literals whose simultaneous truth caused the conflict. To prevent the solver from ever repeating this exact combination, we add a clause that forbids it: at least one of these literals must be false next time. “At least one of $C_1, \dots, C_n$ is false” is written as $\neg C_1 \vee \dots \vee \neg C_n$.

Type 2 — Backjump level

Assignment

After a 1-UIP clause is learned at decision level K , to which level does CDCL(T) backjump?

Source: exam-2026-07-a.pdf (multiple-choice quiz)

Solution

Reasoning. The learned clause becomes unit (and can propagate) as soon as all its literals but one are falsified. Its literals span several decision levels; level K itself is where the conflict was just found, so we don’t stay there. The *next* level at which the clause would already be unit is the second-highest decision level among its literals — backjumping there (rather than only one level, as chronological backtracking would) lets the solver skip the levels in between, since nothing there was relevant to this conflict.

Type 3 — Full worked CDCL(T) conflict example (1-UIP, conflict set, backjump)

Assignment

Let $(h(a)=h(c) \vee p) \wedge (a=b \vee \neg p \vee a=d) \wedge (a \neq d \vee a=b)$ be part of a formula φ , with a prior decision $c = b$ (at level $k - 1$). At level k , the solver proceeds as follows:

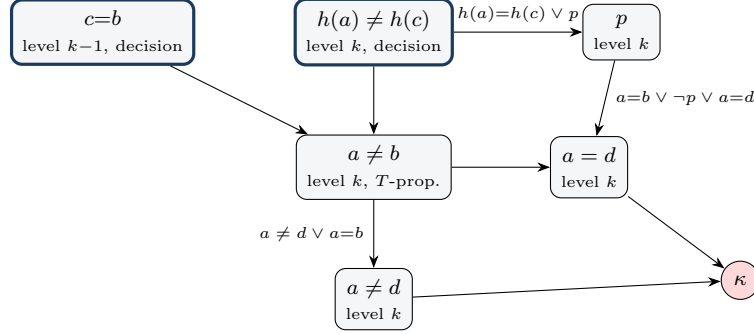
- Decide $h(a) \neq h(c)$
- UnitPropagate p , due to clause $h(a)=h(c) \vee p$
- T -propagate $a \neq b$, since $\{c=b, h(a) \neq h(c)\} \models_T a \neq b$
- UnitPropagate $a = d$, due to clause $a=b \vee \neg p \vee a=d$
- UnitPropagate $a \neq d$, due to clause $a \neq d \vee a=b$
- **Conflict:** $a \neq d$ and $a = d$

Write the implication graph, the 1-UIP, and the conflict set; state the learned clause and the backjump level.

Source: 2SMTEnc.pdf (CDCL(T) implication-graph exercise)

Solution

Implication graph.



$c=b$ (level $k-1$) and the decision $h(a) \neq h(c)$ (level k) jointly T -imply $a \neq b$; $a \neq b$ together with p (itself implied by the decision) forces $a = d$ via the second clause; $a \neq b$ alone forces $a \neq d$ via the third clause; $a = d$ and $a \neq d$ collide at the conflict node κ . All new nodes at level k trace back through the single decision literal $h(a) \neq h(c)$, which is why it is itself the 1-UIP.

1-UIP. Since every derivation path at level k passes through the decision node itself, the 1-UIP is $h(a) \neq h(c)$ (the decision literal itself, in this case).

Conflict set. $\eta = \{h(a) \neq h(c), c = b\}$ — the 1-UIP plus the earlier-level literal it was combined with (via the theory step) to reach the conflict.

Learned clause. The negation of the conflict set: $h(a) = h(c) \vee c \neq b$.

Backjump level. To the highest decision level $< k$ containing a literal of η — here $c = b$ is at level $k-1$, so the solver backjumps to level $k-1$ (the second-highest level appearing in η).

Type 4 — Richer worked example: CDCL($T_{\mathbb{Z}}$) with linear integer arithmetic

Assignment

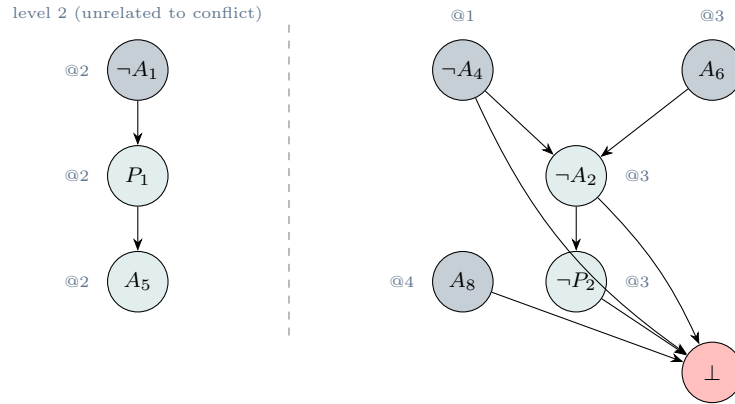
Let φ (over linear integer arithmetic literals, abstracted to Booleans $A_1, \dots, A_8, P_1, P_2$) include clauses $c_1, \dots, c_8$, and suppose the solver reaches the trail

$$M = [\neg A_4, \neg A_1, P_1, A_5, A_6, \neg A_2, \neg P_2, A_8]$$

with decision levels $\neg A_4@1, \neg A_1@2, P_1@2, A_5@2$ (a separate propagation chain), $A_6@3, \neg A_2@3, \neg P_2@3, A_8@4$. The arithmetic literals underlying $\neg A_2$ and A_8 combine (via $\neg(3x_1 - x_3 \leq 6)$ and $\neg(x_1 - x_5 \leq 1)$) into a T -conflict $\perp$. Write the implication graph, the 1-UIP, and the conflict set(s); state the backjump level.

Source: 2SMTEnc.pdf (CDCL($T_{\mathbb{Z}}$) example, originally from a CAV Verification Mentoring Workshop 2017 talk by Alberto Griggio, FBK Trento)

Solution



1-UIP. A_8 (level 4) — the only node at the highest level, dominating all conflict-level derivation paths.

Conflict sets. Two sound choices exist: $\eta' = \{A_8, \neg A_2, \neg A_4\}$ (the raw 1-UIP conflict set) and $\eta'' = \{A_8, A_6, \neg A_4\}$ (obtained by resolving away $\neg A_2$ using its antecedent clause, i.e. replacing it with what implied it, A_6 and $\neg A_4$).

Backjump level. Both learned clauses become unit and propagate $\neg A_8$ once we backjump to **level 3** (the highest level below 4 that appears in either η' or η'' , via $\neg A_2$ or A_6). Note that level 2 (the $\neg A_1, P_1, A_5$ chain) does not contribute to the conflict at all — it is simply irrelevant trail history that gets skipped.

Type 5 — Why theory-conflict clauses are added (quiz)

Assignment

Why is a theory-conflict clause added to the Boolean search in CDCL(T)? (a) to prevent the SAT solver from finding different Boolean assignments implying the same theory-inconsistent combination; (b) to replace the current theory solver with a propositional encoding of the theory; (c) to mark the current Boolean abstraction as theory-consistent; (d) to convert the conflicting theory atoms into branching priorities.

Source: exam-2026-07-b.pdf (multiple-choice quiz)

Solution

Reasoning. This is exactly what the “blocking clause” did in the lazy SMT loop worked example above: after finding $M = \{\ell_1, \neg \ell_2, \neg \ell_4\}$ was T -inconsistent, we added $\neg \ell_1 \vee \ell_2 \vee \ell_4$ specifically so the SAT solver could never again propose a Boolean assignment containing that same inconsistent combination. (b),(c),(d) all describe things a theory-conflict clause does *not* do — it doesn’t replace the theory solver, doesn’t claim consistency (quite the opposite), and doesn’t affect branching priorities directly.

Answer: (a). The clause encodes “never again pick this exact combination of theory-inconsistent literals” at the Boolean level, forcing the SAT search to explore genuinely different territory.

Type 6 — Numeric 1-UIP backjump example (quiz)

Assignment

A 1-UIP learned clause is derived at decision level 9. It contains one literal assigned at level 9, and the remaining at levels 0, 3, 4, and 7. To which level does CDCL(T) backjump? (a) level 8, because backjumping returns to the previous decision level; (b) level 7, the highest decision level below 9 occurring in the learned clause; (c) level 0, the lowest level occurring in the learned clause; (d) level 3, because it is the lowest nonzero decision level occurring in the learned clause.

Source: *exam-2026-07-b.pdf* (multiple-choice quiz)

Solution

Reasoning. Recall the rule (from the tips box above): backjump to the *second-highest* decision level appearing in the learned clause — *not* simply “one level back” (that would be chronological backtracking, ruling out (a), which isn’t even correct here since level $9-1=8$ doesn’t even appear among the clause’s levels), and not the lowest level either (ruling out (c) and (d), which would throw away useful trail history unnecessarily).

Step — apply the rule. The clause’s literal levels are $\{9, 0, 3, 4, 7\}$. Excluding the current conflict level (9), the remaining levels are $\{0, 3, 4, 7\}$; the highest among these is 7.

Answer: (b). Level 7 — backjumping there makes the learned clause immediately unit (only the level-9 literal is left unassigned after undoing everything above level 7), letting the solver skip levels 8 and 9 entirely without losing any relevant trail history.

4 LP / ILP / MIP

4.1 Modelling

How to approach it:

- Structure exactly as for CP: **Variables**, **Constraints**, **Objective**, each commented.
- Linearizing common logical patterns (all with binary $y_i \in \{0, 1\}$):
 - Mutual exclusion (“not both A and B ”): $y_A + y_B \leq 1$.
 - Implication (“if A then B ”): $y_A \leq y_B$.
 - Conjunction bonus / AND variable ($z = y_A \wedge y_B$): $z \leq y_A$, $z \leq y_B$, $z \geq y_A + y_B - 1$, plus benefit term + bonus $\cdot z$ in the objective.
- Only linear constraints/objective are allowed — if the exam says so explicitly, do not use products of two variables directly; introduce an auxiliary linearized variable as above.

Type 1 — MIP formulation with logical constraints

Assignment

A system can enable three optional features: RAG (benefit 8, GPU cost 3), Code Execution (benefit 10, cost 4), Vision (benefit 12, cost 5). GPU budget is 8. Rules: Code Execution and Vision cannot both be selected; if Vision is selected then RAG must also be selected; if both RAG and Code Execution are selected, an additional benefit of 3 is gained. Formulate a MIP maximizing total benefit.

Source: exam-2026-07-a.pdf (Exercise 2, part i)

Solution

Step 1 — decision variables. One binary per feature: $y_R, y_{CE}, y_V \in \{0, 1\}$ (1 if selected, 0 otherwise).

Step 2 — translate “cannot both be selected”. At most one of y_{CE}, y_V can be 1: $y_{CE} + y_V \leq 1$.

Step 3 — translate “if Vision then RAG”. This is a logical implication $y_V \Rightarrow y_R$. In linear form, an implication “if A then B ” (both binary) becomes $A \leq B$: if $A = 1$, this forces $B \geq 1$, i.e. $B = 1$; if $A = 0$, no constraint. So: $y_V \leq y_R$.

Step 4 — translate the conditional bonus. We want an extra +3 in the objective, but *only* when both $y_R = 1$ and $y_{CE} = 1$. We can’t write $y_R \cdot y_{CE}$ directly (not linear), so introduce an auxiliary variable z meant to equal $y_R \wedge y_{CE}$, and force this meaning with three linear constraints:

$$z \leq y_R, \quad z \leq y_{CE}, \quad z \geq y_R + y_{CE} - 1$$

Check this works: if $y_R = y_{CE} = 1$, the third constraint forces $z \geq 1$, and combined with $z \leq 1$ (from either of the first two, since z is binary), $z = 1$. If either is 0, the first or second constraint forces $z \leq 0$, so $z = 0$. Exactly the AND behavior.

Step 5 — the GPU budget. $3y_R + 4y_{CE} + 5y_V \leq 8$.

Full model.

$$\begin{aligned}
y_{CE} + y_V &\leq 1 && \text{(mutual exclusion)} \\
y_V &\leq y_R && \text{(implication: Vision} \Rightarrow \text{RAG)} \\
z &\leq y_R, \quad z \leq y_{CE}, \quad z \geq y_R + y_{CE} - 1 && \text{(linearized AND)} \\
3y_R + 4y_{CE} + 5y_V &\leq 8 && \text{(GPU budget)} \\
\max \quad &8y_R + 10y_{CE} + 12y_V + 3z
\end{aligned}$$

Type 2 — MIP with a conditional (activation-dependent) cost**Assignment**

A bakery produces cakes and biscuits, in integer quantities. Each cake gives profit 5, each biscuit profit 4. At most 10 units of each can be produced. Total production capacity is 12 units. The bakery may activate an extra oven, which increases capacity by 5 units; when activated, each produced unit incurs an extra cost of 2 per cake and 1 per biscuit. Formulate as a MIP maximizing total profit, using only linear constraints/objective.

Source: 7LPext.pdf (bakery MIP exercise)

Solution

Step 1 — decision variables. $C, B \in \{0, \dots, 10\}$: integer number of cakes/biscuits produced. $y \in \{0, 1\}$: whether the extra oven is activated.

Step 2 — the capacity constraint. Normal capacity is 12, extended by 5 if the oven is active: $C + B \leq 12 + 5y$ (when $y = 0$, this is $C + B \leq 12$; when $y = 1$, it's $C + B \leq 17$ — exactly what we want).

Step 3 — the hard part: a cost that only applies when $y = 1$. The extra cost is 2 per cake +1 per biscuit, but *only* if $y = 1$ — that's the product $y \cdot (2C + B)$, which is not linear (a binary times a continuous/integer variable).

Step 4 — linearize the product with auxiliary variables. Introduce u to stand for $y \cdot C$ (i.e., “ C if the oven is active, else 0”) and v for $y \cdot B$. Force this meaning with the standard big-M-style linearization (using each variable's known upper bound, 10):

$$u \leq C, \quad u \leq 10y, \quad u \geq C - 10(1 - y) \qquad v \leq B, \quad v \leq 10y, \quad v \geq B - 10(1 - y)$$

Check: if $y = 1$, then $u \leq 10$ is not binding, and $u \geq C - 0 = C$ combined with $u \leq C$ forces $u = C$ exactly. If $y = 0$, then $u \leq 10 \cdot 0 = 0$ forces $u = 0$ regardless of C . Exactly the desired product.

Step 5 — objective. Ordinary profit $5C + 4B$, minus the extra cost *only when active* — which is now the linear term $2u + v$ (since u, v already encode “times y ”):

$$\max \quad 5C + 4B - 2u - v$$

Full model.

$$\begin{aligned}
C + B &\leq 12 + 5y && \text{(capacity, extended by 5 if oven active)} \\
u &\leq C, \quad u \leq 10y, \quad u \geq C - 10(1 - y) && \text{(linearized } u = y \cdot C) \\
v &\leq B, \quad v \leq 10y, \quad v \geq B - 10(1 - y) && \text{(linearized } v = y \cdot B) \\
\max \quad &5C + 4B - 2u - v
\end{aligned}$$

Type 3 — Describing how a MIP solver works

Assignment

Briefly describe, in natural language, how a modern MIP solver would solve the model above (max. 10 lines).

Source: exam-2026-07-a.pdf (Exercise 2, part ii)

Solution

The solver first relaxes integrality, solving the continuous LP relaxation via the simplex method to obtain an upper bound (for maximization). If the LP optimum is already integral, it is the MIP optimum. Otherwise, it applies *branch-and-bound*: it selects a fractional variable and branches into two subproblems (e.g. $y_i = 0$ / $y_i = 1$), recursively solving the LP relaxation of each; branches whose LP bound is no better than the best integer solution found so far are pruned. Modern solvers additionally strengthen the relaxation with *cutting planes* (e.g. Gomory cuts, cover cuts, clique cuts) derived from the problem structure, tightening bounds before or during branching. The search continues until every branch is either pruned, infeasible, or yields an integral solution, and the best integral solution found is returned as optimal.

4.2 Simplex, Gomory Cuts, and Branch-and-Bound (numeric)

How to approach it:

- **Simplex step:** put the LP in standard form (add slack variables), pick an entering variable (most attractive reduced cost), run the ratio test on the RHS/coefficient to find the leaving variable, pivot.
- **Reading off the bound:** after pivoting, the current objective value is a valid *upper* bound (for a maximization problem) on the true integer optimum, since the LP relaxation always dominates the ILP.
- **Gomory cut:** for a basic variable $x_k^* = \beta_k + \sum_j \alpha_{k,j} x_j$ with fractional β_k , take fractional parts $f_k = \beta_k - \lfloor \beta_k \rfloor$ and $f_{k,j}$ of the coefficients; the cut is $\sum_j f_{k,j} x_j \geq f_k$ (equivalently written as $-f_k + \sum_j f_{k,j} x_j \geq 0$).
- **Branch-and-bound decision:** if the current LP bound is *no better* than the best integer solution found so far (the incumbent), prune the branch; otherwise keep branching (split on a fractional variable, e.g. $x_i \leq \lfloor x_i^* \rfloor$ vs. $x_i \geq \lceil x_i^* \rceil$).

Type 1 — Full worked exercise: pivot, cut, and B&B decision

Assignment

Consider $\max z = 3x_1 + x_2$ s.t. $2x_1 + x_2 \leq 5$, $x_1 + 3x_2 \leq 9$, $x_1, x_2 \geq 0$, $x_1, x_2 \in \mathbb{Z}$. **(1)** Reduce to standard form and apply one simplex pivot to the LP relaxation; give the basic solution and an upper bound on z . **(2)** Define the Gomory cut for the variable with fractional value. **(3)** Given an incumbent bound $z \geq k$, what is the next B&B step if $k < 7$? If $k = 7$?

Source: 6ILP.pdf (worked exercise, simplex/Gomory cut/branch-and-bound)

Solution

(1a) Standard form. Add one slack variable per inequality to turn $\leq$ into $=$:

$$2x_1 + x_2 + s_1 = 5, \quad x_1 + 3x_2 + s_2 = 9, \quad s_1, s_2 \geq 0$$

(1b) Starting basic solution. With no better information, start from the “do nothing” solution: $x_1 = x_2 = 0$ (non-basic), so $s_1 = 5$, $s_2 = 9$ (basic), giving $z = 0$.

(1c) Choose the entering variable. Look at the objective coefficients: x_1 has coefficient 3, x_2 has coefficient 1. Pick the larger one, x_1 , as it improves z fastest per unit increase.

(1d) Ratio test to find the leaving variable. For each constraint row, divide the RHS by x_1 's coefficient in that row (only where the coefficient is positive): row 1 gives $5/2 = 2.5$; row 2 gives $9/1 = 9$. Take the *smallest* ratio (this is the largest x_1 can grow before some variable would go negative): 2.5, from row 1 — so s_1 leaves the basis.

(1e) Pivot. Solve row 1 for the entering variable x_1 , then substitute into row 2 and the objective:

$$x_1 + 0.5x_2 + 0.5s_1 = 2.5 \quad 2.5x_2 - 0.5s_1 + s_2 = 6.5$$

(1f) Read off the new basic solution. Non-basic variables (x_2, s_1) are 0; basic variables take the RHS values: $x_1 = 2.5$, $s_2 = 6.5$.

(1g) Check optimality and get the bound. Substitute $x_1 = 2.5 - 0.5x_2 - 0.5s_1$ into the objective $z = 3x_1 + x_2$:

$$z = 3(2.5 - 0.5x_2 - 0.5s_1) + x_2 = 7.5 - 0.5x_2 - 1.5s_1$$

Both remaining coefficients (-0.5 for x_2 , -1.5 for s_1) are ≤ 0 — increasing either non-basic variable would only *decrease* z , so this is already the LP optimum: $z^{LP} = 7.5$. Since the true (integer) z can be at most this, and must be an integer: $z \leq 7$.

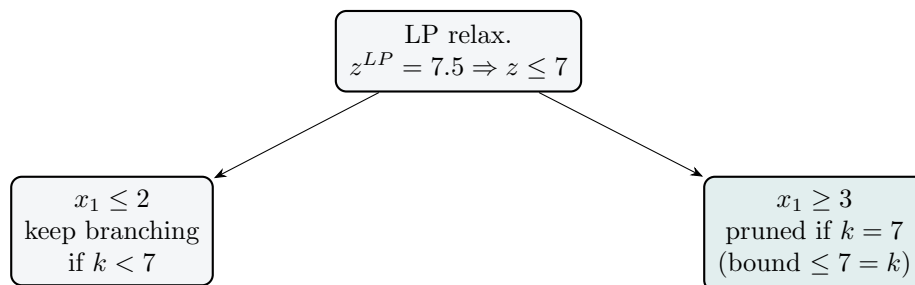
(2) Deriving the Gomory cut. Take the row where the fractional basic variable ($x_1 = 2.5$) was pivoted: $x_1 + 0.5x_2 + 0.5s_1 = 2.5$. Split 2.5 into integer + fractional part: $\beta = 2.5 = 2 + 0.5$, so $f = 0.5$. The coefficients of the non-basic variables (0.5, 0.5) are already fractions in $[0, 1)$. The Gomory cut says: the fractional parts of the non-basic coefficients, weighted by those variables, must sum to at least the fractional part f :

$$0.5x_2 + 0.5s_1 \geq 0.5 \quad \text{i.e. (multiplying by 2)} \quad x_2 + s_1 \geq 1$$

(This cut excludes the current fractional point $x_1 = 2.5$ while keeping every integer-feasible point.)

(3) Branch-and-bound decision. We know from (1g) that no integer-feasible solution can exceed $z = 7$ (the LP bound, rounded down since z is integer).

- **If $k < 7$** (e.g. $k = 6$): the bound (7) is still strictly better than what we already have (k), so there's still hope of finding something better here — pruning is *not* justified. Continue branching, e.g. split into $x_1 \leq 2$ and $x_1 \geq 3$ subproblems.
- **If $k = 7$** : the incumbent already achieves the best possible bound for this branch — no integer solution here can beat or even match a strictly better value, so this branch can be **pruned** immediately, without further work.



4.3 Duality and Sensitivity

How to approach it:

- **Finding the dual:** for primal $P : \max(cx)$ s.t. $Ax = b, x \geq 0$ ($b \in \mathbb{R}^m, x \in \mathbb{R}^n$), the dual is $D(P) : \min(by)$ s.t. $A^t y \geq c, y \in \mathbb{R}^m$ — one dual variable y_i per primal *constraint*, one dual constraint per primal *variable* x_j : $\sum_i a_{j,i} y_i \geq c_j$.
- **Weak duality:** for a primal maximization and its dual minimization, every dual feasible value is an upper bound on every primal feasible value (and vice versa framed the other way).
- **Strong duality:** *when both* primal and dual have optimal solutions (both feasible, both bounded), their optimal objective *values* are exactly equal (not just bounded) — this is a strictly stronger statement than weak duality, which only gives an inequality.
- **Sensitivity analysis:** studies how perturbing the input data (costs, right-hand sides) affects feasibility or optimality of the current solution — not about enumerating integer solutions or partitioning the feasible region.
- Infeasibility of the primal does *not* generally correspond one-to-one with infeasibility of the dual (be careful with this common quiz trap: primal infeasible can mean dual unbounded or infeasible, depending on the case).

Type 1 — Finding the dual of a primal LP

Assignment

Find the dual of: $\max x_1 + x_2$ s.t. $3x_1 + 2x_2 + x_3 = 5$ (y_1), $4x_1 + 5x_2 + x_4 = 4$ (y_2), $x_2 + x_5 = 2$ (y_3), $x_1, \dots, x_5 \geq 0$.

Source: 5LP.pdf (“find the dual” worked exercise)

Solution

Step 1 — one dual variable per primal constraint. There are 3 primal constraints (already labelled y_1, y_2, y_3 in the assignment) — these become the 3 dual variables.

Step 2 — the dual objective uses the primal’s right-hand sides. The primal constraints have RHS 5, 4, 2; the dual (a minimization, since the primal is a maximization) uses these as its objective coefficients:

$$\min 5y_1 + 4y_2 + 2y_3$$

Step 3 — one dual constraint per primal variable. For each primal variable x_j , read down its *column* of coefficients across the three primal constraints, and build a dual constraint $\geq$ the primal’s objective coefficient for x_j .

- x_1 ’s column: coefficient 3 in constraint 1, coefficient 4 in constraint 2, coefficient 0 in constraint 3 (absent). Primal objective coefficient of x_1 is 1. Dual constraint: $3y_1 + 4y_2 \geq 1$.
- x_2 ’s column: 2 in constraint 1, 5 in constraint 2, 1 in constraint 3. Objective coefficient 1. Dual constraint: $2y_1 + 5y_2 + y_3 \geq 1$.
- x_3 ’s column: 1 in constraint 1 only (it’s essentially a slack for that row). Objective coefficient 0 (it doesn’t appear in the primal objective). Dual constraint: $y_1 \geq 0$.

- x_4 's column: 1 in constraint 2 only. Dual constraint: $y_2 \geq 0$.
- x_5 's column: 1 in constraint 3 only. Dual constraint: $y_3 \geq 0$.

Full dual.

$$\begin{array}{rcll} & 3y_1 + 4y_2 & \geq 1 & (x_1) \\ & 2y_1 + 5y_2 + y_3 & \geq 1 & (x_2) \\ \min \ 5y_1 + 4y_2 + 2y_3 & \text{s.t.} & y_1 & \geq 0 \quad (x_3) \\ & & y_2 & \geq 0 \quad (x_4) \\ & & y_3 & \geq 0 \quad (x_5) \end{array}$$

Type 2 — Sensitivity analysis — shadow price

Assignment

In a production LP (maximize profit subject to resource constraints $Ax \leq b$), the optimal dual value for the “corn” resource constraint is $y_{\text{corn}} = 1$, for “hops” is $y_{\text{hops}} = 2$, and for “malt” is $y_{\text{malt}} = 0$. Interpret these values, and state how the optimal objective changes if the corn supply increases by one unit (within the range where the current basis stays optimal).

Source: 5LP.pdf (shadow price / sensitivity analysis, brewery example)

Solution

Step 1 — what a shadow price means, in general. The shadow price of a resource is the dual variable value at the optimum: it measures how much the optimal objective would improve per extra unit of that resource — *without re-solving the whole LP from scratch*.

Step 2 — why we can avoid re-solving. If B is the optimal basis, the basic variables are $x_B = A_B^{-1}b$. Perturbing the resource vector to $b + \Delta b$ gives $x_B(b + \Delta b) = A_B^{-1}b + A_B^{-1}\Delta b$. As long as this stays ≥ 0 (the perturbation is small enough that the same set of constraints remains binding), the objective simply shifts by $c_B A_B^{-1}\Delta b = y \cdot \Delta b$ — a direct computation, no new simplex run needed.

Step 3 — interpret each given value.

- $y_{\text{corn}} = 1$: each extra unit of corn is worth \$1 of additional profit.
- $y_{\text{hops}} = 2$: each extra unit of hops is worth \$2 of additional profit.
- $y_{\text{malt}} = 0$: extra malt is worth *nothing* — this tells us the malt constraint is not binding at the optimum (there’s already slack in it), so relaxing it further changes nothing.

Step 4 — answer the specific question. Increasing corn by exactly 1 unit changes the optimal objective by $y_{\text{corn}} \cdot (\Delta b_{\text{corn}}) = 1 \cdot 1 = \1 — provided this increase is small enough not to change which constraints are binding (i.e., within the “sensitivity range” where the current optimal basis stays optimal).

Type 3 — Primal/dual bound relation (quiz-style)

Assignment

Consider a primal maximization problem and its dual minimization problem. Which statement is correct? (a) primal infeasibility $\equiv$ dual infeasibility; (b) primal and dual feasible values always coincide; (c) a primal feasible value is an upper bound on a dual feasible value; (d) a dual feasible value is an upper bound on a primal feasible value.

Source: exam-2026-07-a.pdf (multiple-choice quiz)

Solution

Step 1 — recall weak duality. For a max/min primal/dual pair, weak duality states: every dual-feasible objective value is $\geq$ every primal-feasible objective value. This is because the dual constraints are built precisely to bound the primal objective from above.

Step 2 — check each option against this. (a) is false — primal infeasibility can instead mean the dual is *unbounded*, not necessarily infeasible. (b) is false in general — feasible values only *coincide at the optimum* (strong duality), not for every feasible pair. (c) has the inequality direction backwards.

Answer: (d). A dual-feasible value upper-bounds a primal-feasible value — exactly weak duality, stated correctly.

Type 4 — Strong LP duality (quiz)

Assignment

What does strong LP duality state when both primal and dual have optimal solutions? (a) their optimal objective values are equal; (b) their feasible regions coincide; (c) their simplex bases have the same variables; (d) their Boolean abstractions have the same solutions.

Source: exam-2026-07-b.pdf (multiple-choice quiz)

Solution

Reasoning. (b) is false — the primal lives in x -space and the dual in y -space; they're not even the same kind of object, so "feasible regions coinciding" doesn't make sense. (c) confuses a specific simplex implementation detail with the general duality statement (and isn't even generally true). (d) is a red herring — "Boolean abstraction" belongs to SAT/SMT, not LP duality.

Answer: (a). Strong duality sharpens weak duality (which only gives an inequality, $\text{dual} \geq \text{primal}$) into an *equality* exactly when both problems have optimal solutions: $\max(cx^*) = \min(by^*)$.

Type 5 — Effects of perturbing b (quiz)

Assignment

What can happen when the right-hand side vector b (i.e., the known term) of a linear program in standard form is perturbed? (a) integrality constraints are the affected part; (b) feasibility is fixed; the objective value changes with the perturbation; (c) the perturbation changes slack-variable but leaves the feasible region unchanged; (d) both feasibility and optimality of the current solution may be affected.

Solution

Reasoning. Recall the shadow-price mechanics from the sensitivity exercise above: $x_B(b + \Delta b) = A_B^{-1}b + A_B^{-1}\Delta b$. This is only guaranteed non-negative (i.e., *feasible*) for *small enough* Δb — a large enough perturbation can push some x_B negative, breaking feasibility of the current basis entirely. (a) is wrong (this is a pure LP question, no integrality involved). (b) wrongly assumes feasibility is untouched. (c) is also too narrow — the feasible region itself (defined by $Ax \leq b$) genuinely shifts when b changes, it's not just a bookkeeping change in slacks.

Answer: (d). Depending on the size of the perturbation, both feasibility (does the current basis remain valid?) and optimality (is it still the best basis, or does another basis now dominate?) can be affected — this is precisely why sensitivity analysis needs to establish a *range* for Δb within which the current basis stays both feasible and optimal.